

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

Step 2: Load Dataset

```
df = pd.read_excel('Online Retail.xlsx')

df.head()
df.info()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	InvoiceNo	541909 non-null	object
1	StockCode	541909 non-null	object
2	Description	540455 non-null	object
3	Quantity	541909 non-null	int64
4	InvoiceDate	541909 non-null	datetime64[ns]
5	UnitPrice	541909 non-null	float64
6	CustomerID	406829 non-null	float64
7	Country	541909 non-null	object

dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 33.1+ MB

Step 3: Data Cleaning

```
df = df[df['Quantity'] > 0]
df = df[df['UnitPrice'] > 0]

df.dropna(subset=['CustomerID'], inplace=True)

df['CustomerID'] = df['CustomerID'].astype(int)
```

Step 4: Create Total Amount

```
df['TotalAmount'] = df['Quantity'] * df['UnitPrice']
```

Step 5: Create RFM Features

```
## Recency
snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)

## Create RFM Table
rfm = df.groupby('CustomerID').agg({
    'InvoiceDate':
        lambda x:
            (snapshot_date - x.max()).days,
    'InvoiceNo': 'nunique',
    'TotalAmount': 'sum'
})

rfm.columns = [
    'Recency',
    'Frequency',
    'Monetary'
]
rfm.head()
```

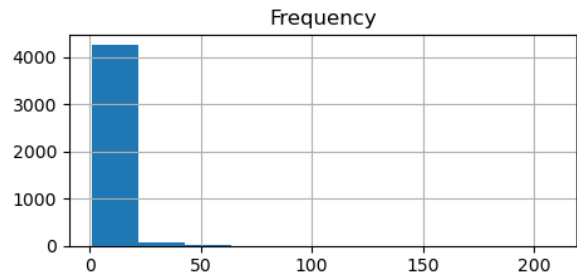
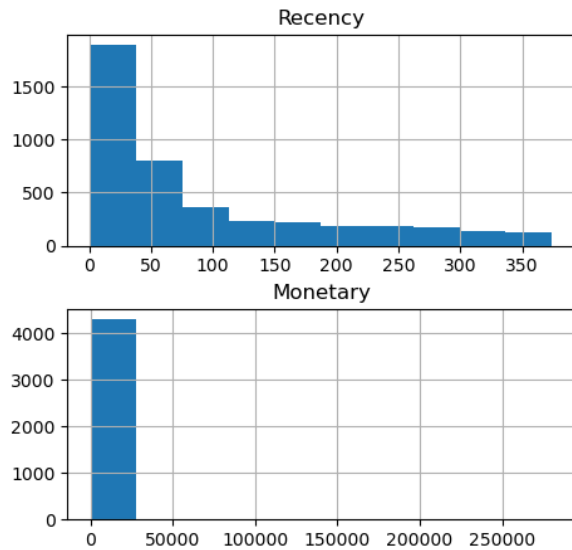
CustomerID	Recency	Frequency	Monetary
12346	326	1	77183.60
12347	2	7	4310.00
12348	75	4	1797.24
12349	19	1	1757.55
12350	310	1	334.40

Step 6: Exploratory Data Analysis

```
rfm.describe()
```

	Recency	Frequency	Monetary
count	4338.000000	4338.000000	4338.000000
mean	92.536422	4.272015	2054.266460
std	100.014169	7.697998	8989.230441
min	1.000000	1.000000	3.750000
25%	18.000000	1.000000	307.415000
50%	51.000000	2.000000	674.485000
75%	142.000000	5.000000	1661.740000
max	374.000000	209.000000	280206.020000

```
##Distribution
rfm.hist(figsize=(12,5))
plt.show()
```



Step 7: Standardization

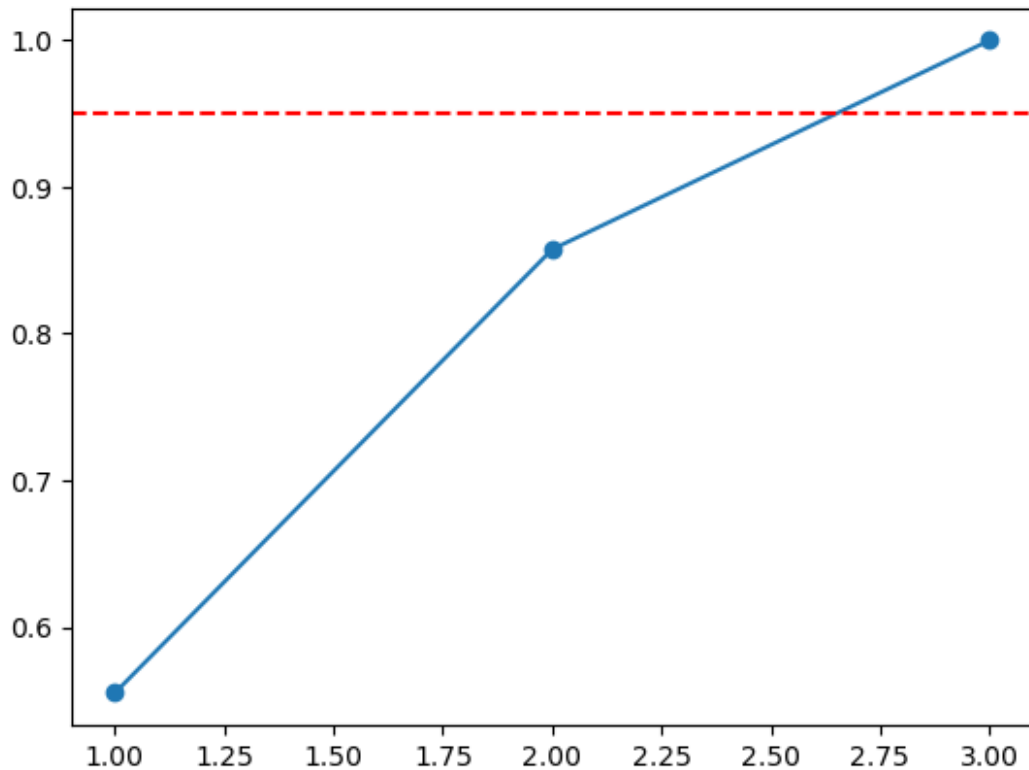
```
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm)
```

Step 8: PCA

```
pca = PCA()
pca_data = pca.fit_transform(rfm_scaled)

## Explained Variance
explained = np.cumsum(
    pca.explained_variance_ratio_
)

plt.plot(
    range(1,
        len(explained)+1),
    explained,
    marker='o'
)
plt.axhline(
    y=0.95,
    color='red',
    linestyle='--'
)
plt.show()
```



```
## Reduce to 2 Components  
pca = PCA(n_components=2)  
  
X_pca = pca.fit_transform(  
    rfm_scaled  
)
```

Step 9: Elbow Method

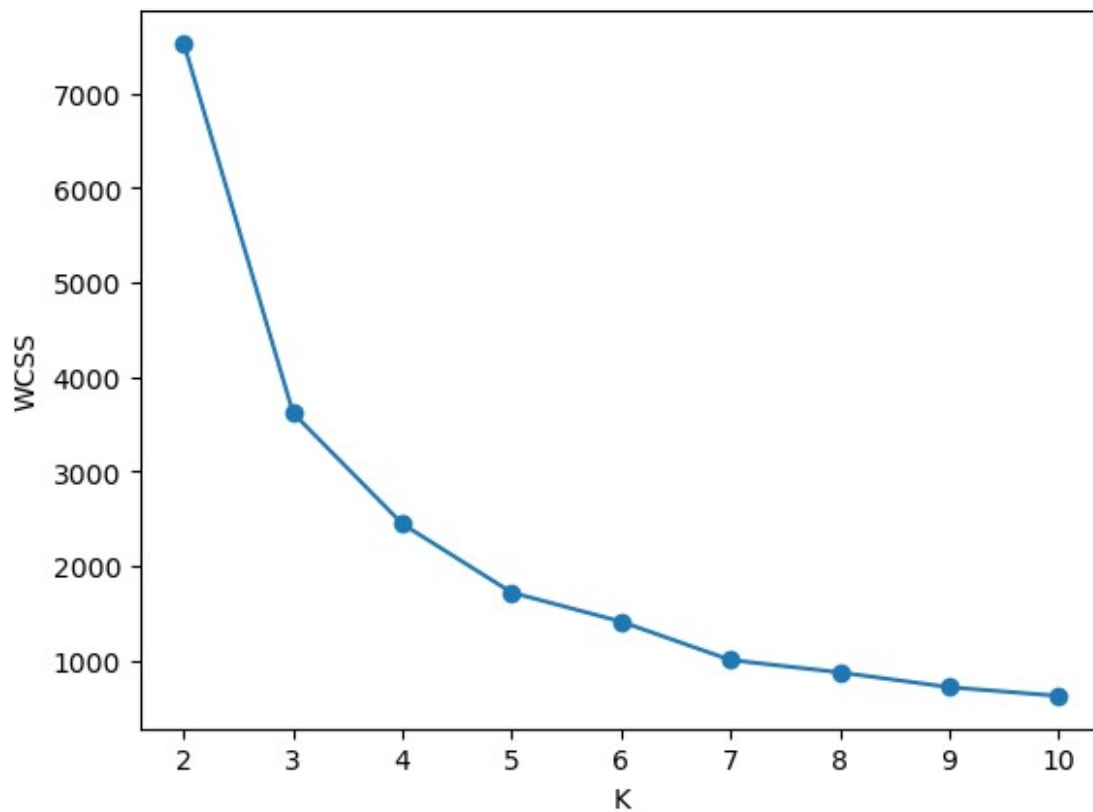
```
wcss = []  
  
for i in range(2,11):  
    kmeans = KMeans(  
        n_clusters=i,  
        random_state=42  
    )  
    kmeans.fit(X_pca)  
    wcss.append(  
        kmeans.inertia_  
    )  
  
plt.plot(  
    range(2,11),
```

```

    wcss,
    marker='o'
)

plt.xlabel('K')
plt.ylabel('WCSS')
plt.show()

```



Step 10: Silhouette Score

```

scores = []

for i in range(2,11):
    kmeans = KMeans(
        n_clusters=i,
        random_state=42
    )
    labels = kmeans.fit_predict(
        X_pca
    )
    score = silhouette_score(

```

```

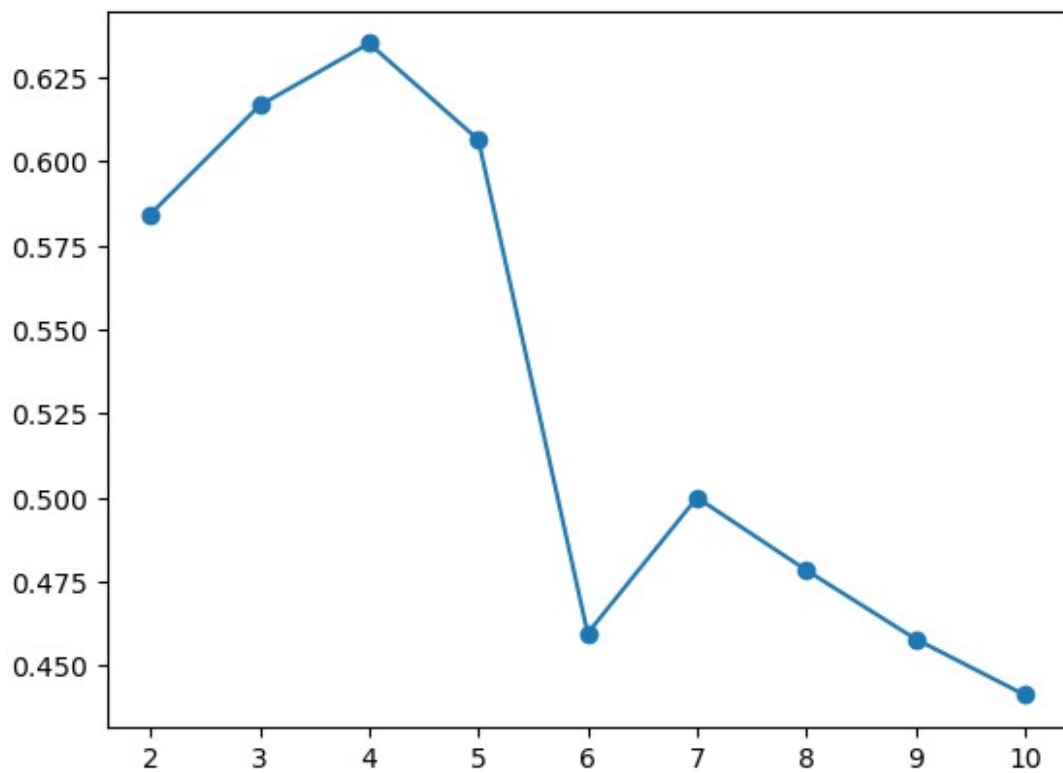
        X_pca,
        labels
    )

    scores.append(score)

plt.plot(
    range(2,11),
    scores,
    marker='o'
)

plt.show()

```



```

best_k = range(2,11)[
    np.argmax(scores)
]

print(best_k)

4

```

Step 11: Final K-Means Model

```
kmeans = KMeans(  
    n_clusters=best_k,  
    random_state=42  
)  
  
rfm['Cluster'] = kmeans.fit_predict(  
    X_pca  
)
```

Step 12: Cluster Visualization

```
plt.figure(figsize=(8,6))  
  
sns.scatterplot(  
    x=X_pca[:,0],  
    y=X_pca[:,1],  
    hue=rfm['Cluster'],  
    palette='Set1'  
)  
  
plt.title(  
    'Customer Segments'  
)  
  
plt.show()
```



Step 13: Customer Personas

```
persona = rfm.groupby(
    'Cluster'
).mean()
```

persona

	Recency	Frequency	Monetary
Cluster			
0	43.283863	3.821776	1392.858624
1	247.756075	1.542056	479.202861
2	7.750000	86.833333	130361.308333
3	15.629412	23.800000	14917.431941